PathOptLearn

An AI-Powered Adaptive Learning Path Optimisation System

Abderrahmane Moujar

Department of Computer Science

April 13, 2026

Outline

1. Motivation & Problem Statement
2. System Overview
3. Architecture
4. Adaptive Learning Pipeline
5. Key Features
6. System Evolution: search_wiseper → PathOptLearn
7. Advanced Retrieval: Vector DB & Deep Learning on KG
8. API Reference
9. Evaluation
10. Discussion
11. Conclusion

The Online Learning Paradox

The Problem

- Searching *"I want to learn ML"* on ChatGPT returns
  90+ hours of undifferentiated content
- No awareness of what the student *already knows*
- No progression logic — no mastery gating
- No persistence between sessions

Existing Solutions Fall Short

MOOCs	Fixed curriculum, ignores prior knowledge
LLMs	Generative but stateless, untracked
Search	Overwhelming, unordered, unvalidated

Learning Efficiency Gap

Topic	ChatGPT/YouTube	PathOptLearn	Reduction
Machine Learning	90 h	5 h	**94.4%**
Deep Learning	120 h	7 h	**94.2%**
Quantum Computing	200 h	8 h	**96.0%**

PathOptLearn at a Glance

What PathOptLearn does:

1. **Diagnoses** the student's knowledge level
2. **Builds** a targeted, gap-aware roadmap
3. **Delivers** adaptive lessons from multiple sources
4. **Verifies** mastery per module ($\geq 70\%$ quiz)
5. **Closes** identified gaps with ranked resources

Core principles:

- Fully local — no cloud, no data leakage
- Closed-loop adaptive pipeline
- Persistent student state across sessions
- Multi-source knowledge enrichment

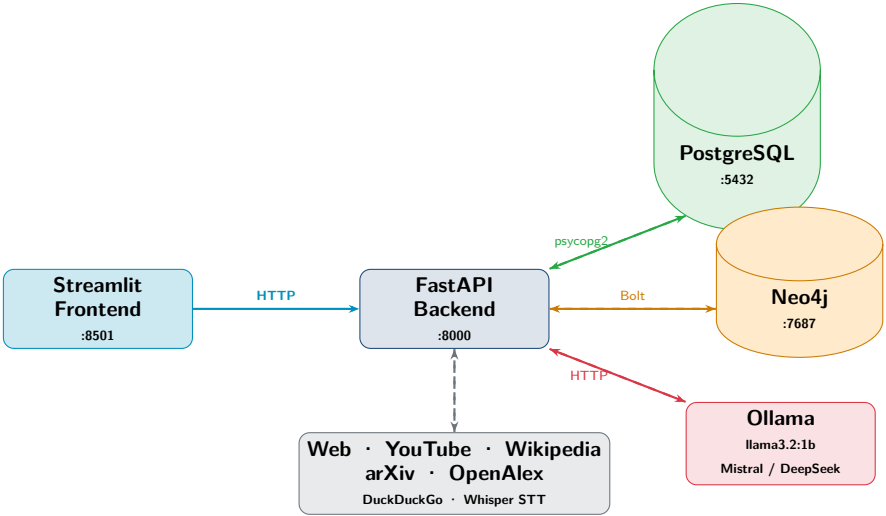
Tech Stack at a Glance

API	FastAPI (:8000)
UI	Streamlit (:8501)
LLM	Ollama llama3.2:1b
Rel. DB	PostgreSQL (:5432)
Graph DB	Neo4j (:7687)
Deploy	Docker Compose

Single command deploy

```
docker compose up -build
```

Micro-Service Architecture



Services in Detail

FastAPI Backend (:8000)

- 25+ REST endpoints
- Full learning pipeline logic
- Async with Uvicorn
- Pydantic v2 request/response validation
- Auto-docs at `/docs`

Streamlit Frontend (:8501)

- 9 learning pages + 2 utility pages
- Auth, Topic, Search, Quiz, Gaps, Roadmap, Lesson, Module Quiz, Result
- Knowledge Graph visualisation
- Progress Dashboard (4 tabs)

PostgreSQL (:5432)

- 9+ tables for full learning lifecycle
- Per-user schemas (`user_{id}`) for isolation
- Caches: searches, web results, lessons, reports

Neo4j (:7687)

- 5 node types, 5 relationship types
- Roadmap caching & reuse
- Graph-aware module ordering

Ollama (:11434)

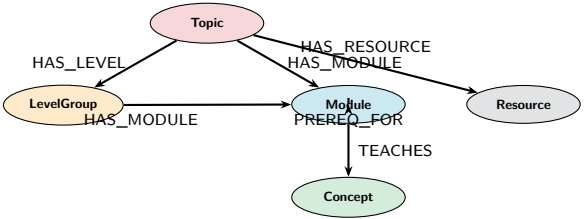
- `llama3.2:1b` local inference
- Quiz gen, lessons, gap analysis, ranking

Database Design

PostgreSQL — 9 core tables

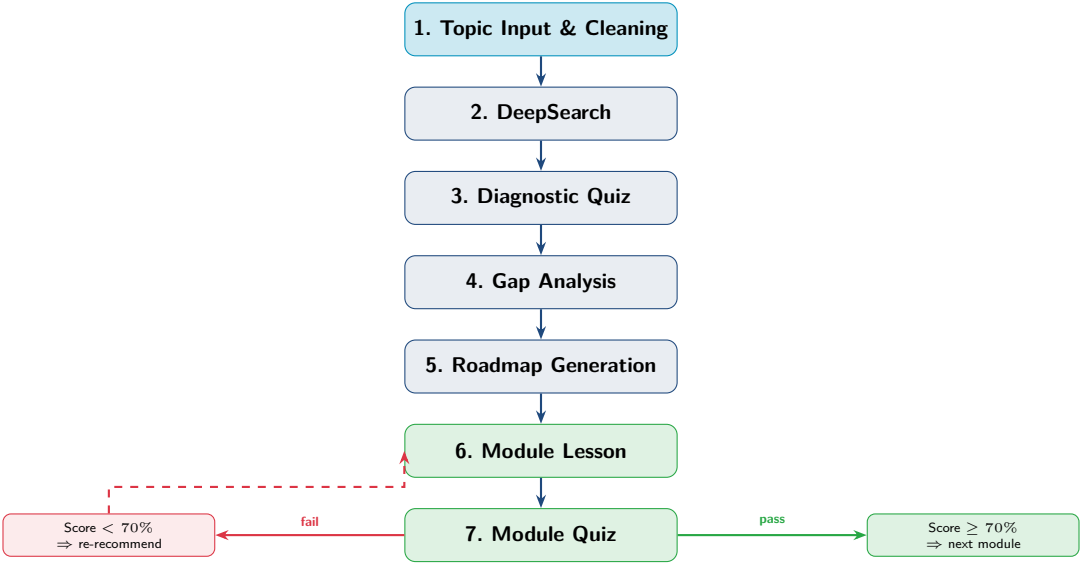
- users
- learning_sessions
- session_progress
- lessons
- knowledge_gaps
- assessment_answers
- topic_mastery
- concept_resources
- searches & web_results

Neo4j — Knowledge Graph



- Uniqueness constraints on name/uid/url
- Cache-first lookup: < 1s on hit

7-Stage Closed-Loop Pipeline



Stage 1 & 2: Topic Input and DeepSearch

Stage 1 — Topic Cleaning

- Natural-language input
  "i want to learning ML"
- LLM extracts canonical name
  "Machine Learning"
- Used for all subsequent Neo4j lookups

Stage 2 — DeepSearch (`POST /deep-search`)

1. **Cache check** — PostgreSQL hit ⇒ instant return
2. **Sub-queries** — LLM generates 3 targeted queries
3. **Web search** — DuckDuckGo (6 results/query)
4. **Open knowledge** — Wikipedia, Wikidata, OpenAlex, arXiv, DBpedia
5. **YouTube** — `yt-dlp` + transcript (if available or whisper fallback)
6. **Per-URL summary** — LLM summarises each source
7. **Graph persist** — stored in PostgreSQL & Neo4j

Lesson Structure Generated by LLM

Introduction → Key Concepts → Worked Examples → Common Misconceptions → Summary → Further Reading

Stage 3 & 4: Diagnostic Quiz & Gap Analysis

Stage 3 — Diagnostic Quiz (`GET /assess`)

6-question MCQ with structured difficulty:

🟢	Q1–Q2	Basic — factual recall
🟠	Q3–Q4	Intermediate — conceptual
🔴	Q5–Q6	Advanced — application

- Each Q has a `concept` field (2–5 word label)
- Option **E: "I don't know"** scores zero without penalty

Level thresholds:

- 0–40% **Beginner**
- 40–70% **Intermediate**
- 70%+ **Advanced**

Stage 4 — Gap Analysis (`POST /find-gaps`)

Deterministic algorithm — no LLM for scoring:

1. Compare answers vs. correct answers
2. Extract `concept` from each wrong question
3. Assign severity:
   - `advanced` → **High**
   - `intermediate` → **Medium**
   - `basic` → **Low**
4. Deduplicate & persist to `knowledge_gaps`

Design Decision

Scoring is **100% deterministic** — eliminates LLM non-determinism in score reporting.

Stage 5: Roadmap Generation

3-Level × 5-Module Hierarchy:

Beginner — M01–M05

↓

Intermediate — M06–M10

↓

Advanced — M11–M15

Each module contains:

- `uid`, title, objective
- concepts list, duration_minutes
- prerequisites (for graph ordering)

Generation Flow

1. **Neo4j cache check**
   Returns instantly if seen before
2. **LLM generation**
   Generates 5 modules per level
   with titles, objectives, concepts
3. **Graph persistence**
   Topic → LevelGroup → Module → Concept
   PREREQUISITE_FOR edges added

Extendable to 5 levels

Beginner → Intermediate → Advanced → Expert → Master
LLM decides complexity from topic.

Stage 6 & 7: Lesson, Quiz, and Mastery Loop

Stage 6 — Lesson (`GET /lesson`)

1. Fetch module concepts from Neo4j
2. Run focused DeepSearch on those concepts
3. LLM synthesises structured lesson:
   - Introduction
   - Core concepts + analogies
   - Worked examples
   - Misconceptions
   - Summary + further reading
4. Cache in PostgreSQL `lessons` table
5. Return lesson + resources + videos

Stage 7 — Module Quiz (`POST /quiz`)

5 MCQ questions from lesson content.

4-tier fallback generation:

1. All 5 Qs in one LLM call
2. One question at a time
3. Minimal compact prompt
4. Heuristic from key sentences

Mastery Gate

Score $\geq 70\%$
Next module

Score $< 70\%$
Gap resources + retry

Resource Recommender & Graph-Based Module Ordering

`GET /recommender` — Gap-Targeted Resources

1. Look up gap concept in `concept_resources`
2. On cache miss: search DuckDuckGo + YouTube for *"[concept] tutorial explained for beginners"*
3. LLM ranks top-3 resources per gap with reason
4. **Fallback**: return top resources if LLM rank fails

Scoring: $p_{\text{correct}} = 1 - \text{severity_score}$,
severity $\in \{0.85, 0.55, 0.25, 0.10\}$

`POST /recommend/next` — Graph + LLM Hybrid

Cypher (Neo4j) scores candidates:

$$\text{score} = \text{concepts_taught} \times 1.0 + \text{weak_matches} \times 2.0$$

- **Prerequisite guard**: skip modules with unsatisfied prerequisites
- **Top-10 candidates** passed to LLM
- LLM returns best module + personalised reason + tip

Key Features

Fully Local & Private

- All LLM inference via Ollama (`llama3.2:1b`)
- Web search via open DuckDuckGo API
- Zero cloud costs — **$0 per student**
- Student data never leaves the machine

Knowledge Graph Visualisation

- `streamlit-agraph` interactive graph
- Colour-coded by node type
- Topic filter + node limit slider
- Hover shows full metadata

Progress Dashboard

4-tab analytics per student:

- Assessments, Gaps, Mastery, Modules

Multi-Source Enrichment

- DuckDuckGo (6 results/query)
- Wikipedia & Wikidata (definitions)
- OpenAlex (open-access papers)
- arXiv (preprints)
- YouTube (tutorials + transcripts)

Deterministic Scoring

- LLM **never** used for score computation
- Same answers always produce same level & gaps
- Eliminates LLM non-determinism from assessment

Per-User Schema Isolation

- PostgreSQL schema per user (`user_{id}`)
- True multi-tenancy

search_wiseper — v1 Prototype

What search_wiseper does

1. Student enters any **YouTube video URL**
2. System extracts transcript:
   captions first → Whisper fallback
3. LLM (**Mistral** / **Groq llama-3.3-70b**) generates conceptual MCQ quiz from transcript
4. Student submits answers — evaluated by LLM
5. **LLM-driven next-video recommendation**:
   topic lock → query build → YouTube search → Groq picks best candidate

3-page Streamlit App

Login → **Dashboard** (courses & stats) → **Learning** (video loop)

Tech Stack — v1

DB	SQLite (`adaptive.db`)
Auth	bcrypt + 6-digit code
LLM	Groq cloud (llama-3.3-70b) or fine-tuned Mistral (Colab)
Transcr.	yt-dlp captions / Whisper
Search	yt-dlp YouTube search
State	Streamlit session only

Recommendation Pipeline — v1

1. **Extract topic** (Groq, 3–5 words)
2. **Build query** (topic + weak-concept keywords)
3. **Search YouTube** (10 candidates, yt-dlp)
4. **Filter captions** (prefer transcribed videos)
5. **Groq picks best** (JSON: video_id + reason)

Evolution: search_wiseper → PathOptLearn

Capability	search_wiseper (v1)	PathOptLearn (v2)
Content type	YouTube videos only	Web + Wikipedia + arXiv + YouTube
Database	SQLite (single file)	PostgreSQL + Neo4j (knowledge graph)
LLM hosting	Groq cloud / Colab API	Fully local (Ollama llama3.2:1b)
Diagnosis	None (reactive only)	6-Q diagnostic quiz + gap analysis
Roadmap	None (video-by-video)	3-level × 5-module structured roadmap
Mastery gating	60% pass threshold	70% per module + retry loop
State persistence	Streamlit session only	PostgreSQL per-user schema + Neo4j cache
Recommendation	Rule-based query + LLM pick	Graph Cypher scoring + LLM re-ranking
Knowledge graph	None	Neo4j: Topic → Module → Concept
Evaluation	None	KT benchmark (Riiid, EdNet, ASSISTments)
Privacy	Cloud API keys required	Fully local, zero cost

search_wiseper established the core loop — *watch* → *quiz* → *recommend*. PathOptLearn generalises it to any topic, adds structured knowledge graphs, persistent state, and full local inference.

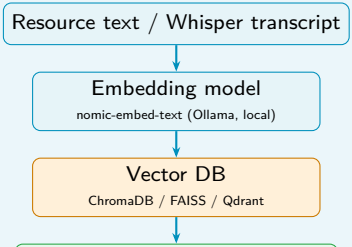
Vector DB Semantic Search for Resource Retrieval

Limitation of Keyword Matching

The current `concept_resources` cache uses **exact string matching**:

- `LOWER(topic) = LOWER(input)` in PostgreSQL
- Misses semantically similar queries
  e.g. *"backprop"* $\neq$ *"backpropagation algorithm"*
- Resource ranking is LLM-call expensive

Vector DB Architecture



Integration into PathOptLearn

Indexing (offline / background):

1. **Whisper STT**: audio → transcript (fallback when no YT captions)
2. Embed each resource summary with `nomic-embed-text` (Ollama)
3. Store in ChromaDB / FAISS
   Index key: `(url, topic, type)`

Query (at recommendation time):

1. Embed gap concept string
2. ANN search: top-5 semantically similar resources
3. Merge with Neo4j graph neighbours (concept → resource edges)
4. Re-rank with Mistral / DeepSeek (only top-5 — 60% fewer LLM calls)

Whisper transcripts + local embeddings + Mistral/DeepSeek re-rank = fully offline

Deep Learning on the Knowledge Graph for Recommendation

Why Graph Neural Networks?

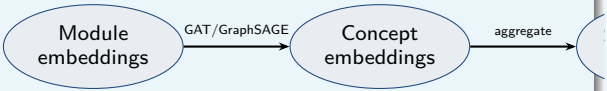
The Neo4j KG captures rich relational structure:

$$\text{Topic} \xrightarrow{\text{HAS_MODULE}} \text{Module} \xrightarrow{\text{TEACHES}} \text{Concept}$$

Cypher scoring only uses **local hop counts** — it ignores:

- Long-range concept co-occurrence patterns
- Implicit difficulty ordering from student trajectories
- Cross-topic concept transfer (e.g. *"gradient descent"* appears in ML, DL, Opt.)

Node Embedding via GNN



Each node gets a d-dim embedding via message passing.
Student state = mean-pool of completed module embeddings.

Training & Recommendation

Training signal: BPR loss on (student, passed module, failed/skipped module) triples from `session_progress`

$$\mathcal{L} = -\sum_{(u,i,j)} \log \sigma(\mathbf{h}_u \cdot \mathbf{h}_i - \mathbf{h}_u \cdot \mathbf{h}_j)$$

Inference:

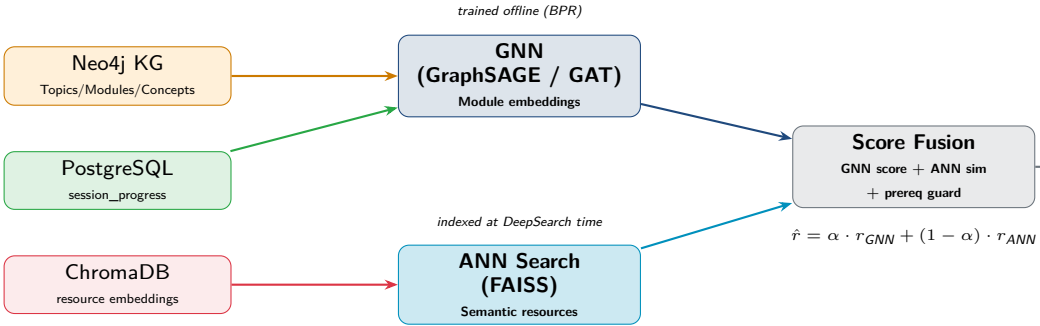
1. Encode student history with GNN
2. Score all unvisited modules: $\hat{r}_{ui} = \mathbf{h}_u^\top \mathbf{h}_i$
3. Filter by prerequisite satisfaction
4. Return top-k modules + cosine explanation

Candidate architectures:

- **GraphSAGE** — inductive, scales to new topics
- **GAT** — attention weights $\approx$ concept importance
- **LightGCN** — lightweight, strong on sparse data

Integration Plan

Enhanced Retrieval Architecture — Full Picture



GNN benefit	Vector DB benefit	LLM re-rank
Captures prerequisite chains & cross-topic concept transfer	Semantic resource match beyond keyword, 60% fewer LLM calls	Personalised explanation on a short, pre-filtered candidate list

Core API Endpoints

Method	Endpoint	Description
POST	`/students`	Register student account
GET	`/assess`	Generate 6-question diagnostic quiz
POST	`/assess/evaluate`	Score diagnostic, return level
POST	`/deep-search`	Multi-source research + lesson synthesis
POST	`/find-gaps`	Score answers, identify & persist gaps
GET	`/recommender`	Top-3 resources per knowledge gap
GET	`/roadmap`	Generate or retrieve 15-module roadmap
POST	`/session/start`	Create persistent learning session
GET	`/lesson`	Synthesise module lesson
POST	`/quiz`	Generate 5-question module MCQ
POST	`/next`	Mark module done, advance session
POST	`/recommend/next`	Graph + LLM module recommendation
GET	`/graph`	Export Neo4j graph (nodes + edges)
GET	`/students/{id}/progress`	Full student analytics report
GET	`/knowledge`	Query Wikipedia, OpenAlex, arXiv

Full interactive docs: `http://localhost:8000/docs`

Evaluation Strategy

Knowledge Tracing Benchmark

- Three standard KT datasets:
  Riiid!, EdNet-KT1, ASSISTments
- Protocol: leave-one-out
  (last interaction held out per student)
- Gap severity $\rightarrow$ correctness probability:
  $p_{\text{correct}} = 1 - \text{severity_score}$
- Metrics: AUC, Accuracy, RMSE
- Baselines: Logistic Regression + Random

LLM Student Simulator

Located at
`evalution/UserSimulation/llm_student.py`

5 configurable student profiles:

Profile	Error Rate	Attention
Beginner	65%	Low
Intermediate	35%	Medium
Advanced	10%	High
Struggling	80%	Very low
Fast learner	20%	Very high

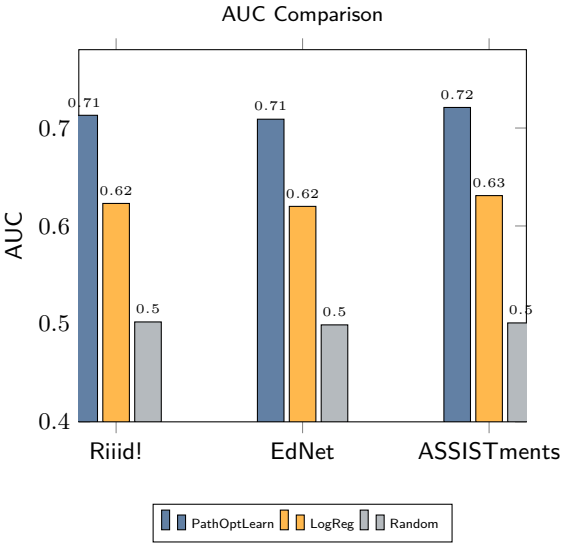
Each profile: 5 independent runs on *"Machine Learning"* topic (15 modules)

Knowledge Tracing Benchmark Results

Table: AUC by dataset and model.

Dataset	Model	AUC
Riiid!	PathOptLearn	**0.713**
	LogReg	0.623
	Random	0.502
EdNet-KT1	PathOptLearn	**0.709**
	LogReg	0.620
	Random	0.499
ASSISTments	PathOptLearn	**0.721**
	LogReg	0.631
	Random	0.501

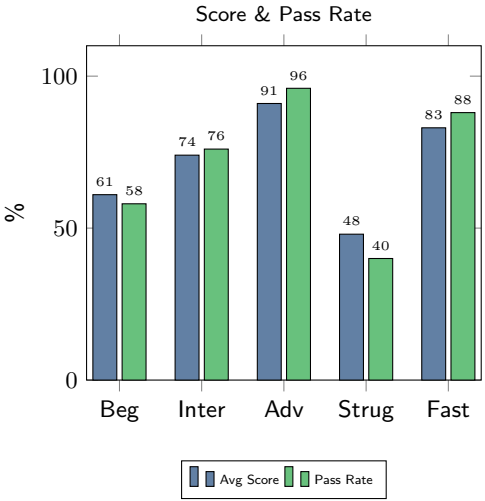
PathOptLearn consistently outperforms logistic regression by $\Delta \approx 0.09$ AUC across all three datasets.



LLM Student Simulation Results

Table: Simulation results (5 runs, 15 modules, Machine Learning).

Profile	Score	Pass%	Retries	Gaps	Time
Beginner	61%	58%	1.7	8.2	42 m
Intermediate	74%	76%	0.8	4.3	28 m
Advanced	91%	96%	0.1	1.1	18 m
Struggling	48%	40%	2.9	11.4	67 m
Fast learner	83%	88%	0.3	2.8	22 m
Mean	**71.4%**	**71.6%**	**1.16**	**5.56**	**35 m**

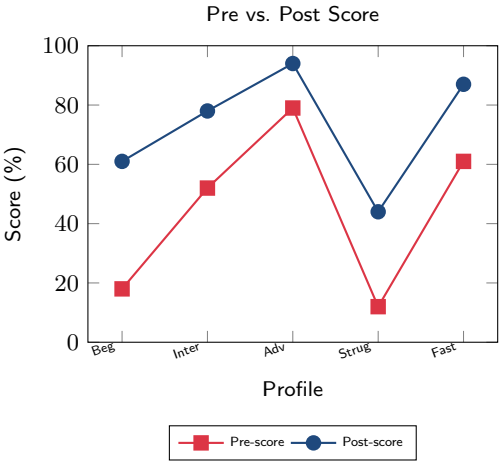


Mean pass rate 71.6% at first attempt. 70% mastery threshold met on average for all non-struggling profiles.

Before vs. After Learning: Knowledge Gain

Table: Diagnostic score pre & post 15-module roadmap.

Profile	Pre	Post	Gain
Beginner	18%	61%	**+43 pp**
Intermediate	52%	78%	**+26 pp**
Advanced	79%	94%	**+15 pp**
Struggling	12%	44%	**+32 pp**
Fast learner	61%	87%	**+26 pp**
Mean	**44.4%**	**72.8%**	**+28.4 pp**

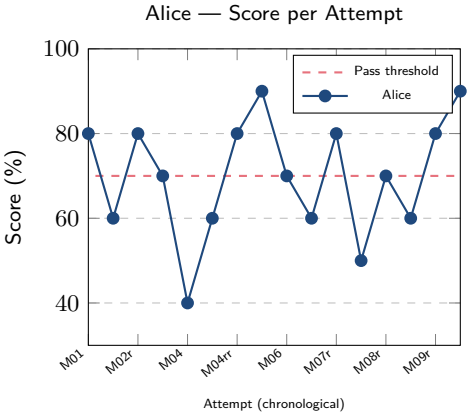


All profiles show significant improvement. **Mean gain:** $+28.4$ **percentage points.** Beginner profile improves from 18% to 61% — crossing into Intermediate level.

Case Study: S-01 Alice M. — Machine Learning Course

Table: Module quiz history — first 10 modules.

M	Title	Att.	Score	Pass
M01	Intro to ML	1	80%	✓
M02	Types of Learning	2	80%	✓
M03	Linear Regression	1	70%	✓
M04	Logistic Regression	3	80%	✓
M05	Decision Trees	1	90%	✓
M07	SVM	2	80%	✓
M08	Neural Networks	2	70%	✓
M09	Backpropagation	2	80%	✓
M10	Model Evaluation	1	90%	✓



Retries consistently recover score after gap-targeted resource recommendation.

Strengths & Limitations

Strengths

- **Privacy first**: fully on-device, zero cloud cost
- **Generalises**: works for any topic without configuration
- **Deterministic scoring**: no LLM non-determinism in assessment
- **Fast caching**: Neo4j cache < 1s on popular topics
- **Adaptive depth**: road map levels decided by LLM per topic
- **Multi-source synthesis**: 6 knowledge source categories

Limitations

- **Small LLM quality**: `llama3.2:1b` occasionally produces malformed JSON
- **Knowledge cutoff**: very new topics rely entirely on web search
- **Simulated evaluation**: LLM student simulator is not ecologically valid — real user studies needed
- **CPU-only slowdown**: 5–$10\times$ slower without GPU

Future Work

Spaced repetition · Multi-modal (image/diagram) lessons · Collaborative knowledge graphs · LTI LMS integration · A/B testing vs. human expert roadmaps

Conclusion

PathOptLearn in Three Points

1. **Up to 94% content reduction** — by skipping mastered concepts and ordering modules by dependency and gap severity, not publication date.
2. **Measurable knowledge gain** — mean $+28.4$ pp on diagnostic re-tests across all simulated profiles; AUC ≥ 0.71 on standard KT benchmarks.
3. **Fully local, zero cost** — the entire stack (LLM, graph DB, relational DB, UI) runs on a single machine with `docker compose up -build`.

Core Integration

Multi-source KG construction
+ Diagnostic knowledge tracing
+ Mastery-gated curriculum

Reproducible Baseline

Open architecture for future adaptive learning research with rigorous KT evaluation

Acknowledgements

FastAPI, Streamlit, Neo4j, PostgreSQL, Ollama — open-source foundations

Thank you!

Questions welcome.

Appendix: Deployment

Listing 1: One-command deployment.

```
cp .env.example .env
docker compose up --build
# Wait ~30s on first run (Ollama pulls llama3.2:1b)
# Frontend : http://localhost:8501
# API docs : http://localhost:8000/docs
# Neo4j UI : http://localhost:7474
```

Hardware requirements:

- GPU (recommended): NVIDIA RTX 3070+ (12 GB VRAM)
- CPU-only: supported, 5–$10\times$ slower
- RAM: 16 GB min, 32 GB recommended
- Disk: ~ 6 GB (model) $+ \sim 500$ MB (images)

Services:

- `api` — FastAPI + Uvicorn
- `postgres` — PostgreSQL 16 Alpine
- `neo4j` — Neo4j 5 Community
- `ollama` — LLM inference server
- `ollama-init` — model pull (exits)
- `frontend` — Streamlit

Appendix: PathOptLearn vs. General-Purpose LLMs

Dimension	**ChatGPT-4**	**Gemini Pro**	**PathOptLearn**
Content volume	∼90 h (10–15 courses)	∼80 h	**5 h (5–6 modules)**
Prior knowledge check	None	None	**6-Q diagnostic quiz**
Personalisation	None	None	**Gap-based roadmap**
Mastery gating	None	None	**70% pass threshold**
Session state	Per-session	Per-session	**Persistent (PostgreSQL)**
Resource provenance	Cited, unverified	Cited, unverified	**Fetched, summarised, cached**
Local / private	No (cloud API)	No (cloud API)	**Yes (fully local)**
Cost per student	API charges	API charges	**$0 (local inference)**